

27.2:

1. Inverted file of documents

Term	Number of Documents	Documents
Auto	2	1,2
Beetle	1	6
Car	2	1,6
Computer	3	2,4,5
Honda	2	1,3
IBM	2	4,5
Manufacturer	2	1,4
Navigation	2	2,3
Personal	1	5
VW	1	6

2. Hash function

Term	Hashed value
Auto	10000
Beetle	01000
Car	00100
Computer	00010
Honda	00001
IBM	10000
Manufacturer	01000
Navigation	00100
Personal	00010
VW	00001

Document	Terms	Hash
1	car, manufacturer, Honda, auto	11101
2	auto, computer, navigation	10110
3	Honda, navigation	00101
4	Manufacturer, computer, IBM	11010
5	IBM, personal, computer	10010
6	car, Beetle, VW	01101

3. From the inverted file:

car -> {1,6}

IBM AND computer:

IBM -> {4,5}

computer -> {2,4,5}

IBM AND computer -> {4,5}

IBM AND car:

IBM -> {4,5}

car -> {1,6}

IBM AND car -> no documents

IBM OR auto:

IBM -> {4,5}

auto -> {1,2}

IBM OR auto -> {1, 2, 4, 5}

IBM AND computer AND manufacturer:

IBM -> {4,5}

computer -> {2,4,5}

IBM AND computer -> {4,5}

manufacturer -> {1,4}

IBM AND computer AND manufacturer -> {4}

From the signature file:

car -> 00100 -> potential matches {1,2,3,6} -> actual matches {1,6}

IBM AND computer:

IBM -> 10000

computer -> 00010

IBM AND computer -> 10010 -> potential matches {2,4,5} -> actual matches {4,5}

IBM AND car:

IBM -> 10000

car -> 00100

IBM AND car -> 10100 -> potential matches: {1,2} -> actual matches: no documents

IBM OR auto:

IBM -> 10000

auto -> 10000

IBM OR auto -> 10000 or 10000 -> potential matches:{1, 2, 4, 5} -> actual matches:{1, 2, 4, 5}

IBM AND computer AND manufacturer:

IBM -> 10000

computer -> 00010

IBM AND computer -> 10010

manufacturer -> 01000

IBM AND computer AND manufacturer -> 11010 -> potential matches {4} -> actual matches {4}

5. a.

Term	IDF
Auto	$\log(6/2) = \log(3) = 0.477$
Beetle	$\log(6/1) = \log(6) = 0.778$
Car	$\log(6/2) = \log(3) = 0.477$
Computer	$\log(6/3) = \log(2) = 0.301$
Honda	$\log(6/2) = \log(3) = 0.477$
IBM	$\log(6/2) = \log(3) = 0.477$
Manufacturer	$\log(6/2) = \log(3) = 0.477$
Navigation	$\log(6/2) = \log(3) = 0.477$
Personal	$\log(6/1) = \log(6) = 0.778$
VW	$\log(6/1) = \log(6) = 0.778$

b.

	auto	Beetle	car	computer	Honda	IBM	manufacturer	navigation	personal	VW
Doc 1	1	0	1	0	1	0	1	0	0	0
Doc 2	1	0	0	1	0	0	0	1	0	0
Doc 3	0	0	0	0	1	0	0	1	0	0
Doc 4	0	0	0	1	0	1	1	0	0	0
Doc 5	0	0	0	1	0	1	0	0	1	0
Doc 6	0	1	1	0	0	0	0	0	0	1

Document vectors

	auto	Beetle	car	computer	Honda	IBM	manufacturer	navigation	personal	VW
Doc 1	0.477	0	0.477	0	0.477	0	0.477	0	0	0
Doc 2	0.477	0	0	0.301	0	0	0	0.477	0	0
Doc 3	0	0	0	0	0.477	0	0	0.477	0	0
Doc 4	0	0	0	0.301	0	0.477	0.477	0	0	0
Doc 5	0	0	0	0.301	0	0.477	0	0	0.778	0
Doc 6	0	0.778	0.477	0	0	0	0	0	0	0.778

c. Query Vectors:

	auto	Beetle	car	computer	Honda	IBM	manufacturer	navigation	personal	VW
Query 1	0	0	0.477	0	0	0	0	0	0	0
Query 2	0	0	0	0.301	0	0.477	0	0	0	0
Query 3	0	0	0.477	0	0	0.477	0	0	0	0

Query 4	0.477	0	0	0	0	0.477	0	0	0	0
Query 5	0	0	0	0.301	0	0.477	0.477	0	0	0

Document	Length
1	0.954
2	0.739
3	0.675
4	0.739
5	0.961
6	1.199

Query	Length
1	0.477
2	0.564
3	0.675
4	0.675
5	0.739

Unnormalized relevance:

	Query 1	Query 2	Query 3	Query 4	Query 5
Doc 1	0.228	0	0.228	0.228	0.228
Doc 2	0	0.091	0	0.228	0.091
Doc 3	0	0	0	0	0
Doc 4	0	0.318	0.228	0.228	0.546
Doc 5	0	0.318	0.228	0.228	0.318

Doc 6	0.228	0	0.228	0	0
-------	-------	---	-------	---	---

Normalized relevance:

	Query 1	Query 2	Query 3	Query 4	Query 5
Doc 1	0.5	0	0.354	0.354	0.323
Doc 2	0	0.217	0	0.457	0.166
Doc 3	0	0	0	0	0
Doc 4	0	0.764	0.457	0.457	1
Doc 5	0	0.587	0.351	0.351	0.448
Doc 6	0.398	0	0.281	0	0

d. One can use the inverted file of documents to identify the top two documents for a given query by using the index to find all documents that include the terms in the query, calculating the TFIDF for each of the terms in the query on each of these documents, calculating the TFIDF of the terms with respect to the query, taking the dot product of each of the document TFIDF vectors with the query TFIDF vector to get the relevance, normalizing each of these relevances by dividing it by the query length times the document length, and selecting the two documents with the highest relevance.

e. If this inverted index is sorted by relevance instead of by document id, you only need to take the top two documents in the list to identify the two documents that best match the query.

f. For each of these queries and documents, the TF for the document would be 10 times higher and the IDF of the document as well as the TF and the IDF of the query would remain the same. Hence, the unnormalized relevance would be 10 times larger for each of the queries for each document, but the normalized relevance would remain the same because the length of the document is 10 times larger.

28.5:

5. Querying for regions that contain the intersection of R1 and R2 would take you into both the R1 and R2 subtrees

6. Querying for regions that contain both R3 and R5 would take us to R3 and R5.

7. Querying for regions that contain both R3 and R4 would take us to R3 and R4 but not to R5 because R5 does not overlap with R3 or R4.

8. No query can do this because there is a nonzero intersection between R3 and R4 so any query that takes you to R3 also takes you to R4.

Ex 27.4

1. If you were given a budget of \$500,000 dollars for purchasing machines, and were required to index all 10 billion pages, could you do it?

If we were given a \$500,000 budget to index all 10 billion pages., we would not be able to do it because \$500,000 will only give us 500 machines and with each machine being capable of indexing 10 million pages, the 500 machines will only be able to index 5 billion pages instead of the 10 billion pages we need to index.

2. What is the minimum budget to index all pages? If you assume that each query can be answered by looking at data in just one (10 million page) partition, and that queries are uniformly distributed across partitions, what peak load (in number of queries per second) can such a cluster handle?

The minimum budget to index all pages will be \$1 million dollars as this will be able to get 1000 machines that be able to index all pages. The cluster can handle 200,000 queries per second.

3. How would your answer to the previous question change if each query, on average, accessed two partitions?

If each query on average accessed two partitions, our answer to the previous question will be 100,000 queries per second since it will take 2 machines to respond to queries that are made.

4. What is the running budget required to handle the desired load of 500 million hits per day if all queries are on a single partition? Assume that queries are uniformly distributed with respect to time of day.

If we are using the 10 billion stored pages and we desire 500 million hits per day load, that means we will be having around 5788 queries per second. To run that many queries, we may need around 29 machines. With each machine being estimated at 1000, the running budget should be \$29,000.

5. Would your answer to the previous question change if the number of queries per day went up to 5 billion hits per day? How would it change if the number of pages went up to 100 billion?

Yes, our answer to the previous question will change significantly if the number of queries per day went up to 5 billion hits per day. The number of queries per second will increase to 57880 which will need at least 290 machines to run. Therefore, the budget will be \$290,000 (290×1000). If the number of pages goes up to 100 billion, then minimum budget will be \$10 million because each machine can handle 10 million pages, and we will need 10,000 machines for that with each machine costing 1000.

6. Assume that each query accesses just one partition, that queries are uniformly distributed across partitions, but that at any given time, the peak load on a partition is up to 10 times the average load. What is the minimum budget for purchasing machines in this scenario?

The minimum budget for this scenario will be \$290,000 if we are to hold all assumptions from previous questions. With 500 million hits per day at 5788 hits per second, the peak will be this value multiply by 10 which will be 57880 and with 290 machines needed and each machine costing 1000, the budget will be \$290,000

7. Take the cost for machines [take the previous question and multiply it by 10 to reflect the costs of maintenance, administration, network bandwidth, etc. This amount is your annual cost of operation. Assume that you charge advertisers 2 cents per page. What fraction of your inventory (i.e., the total number of pages that you serve over the course of a year) do you have to sell in order to make a profit?

**Using the \$290,000 from previous question as cost of machine,
Number of ads pages = $\$290,000 / 0.02 = 14,500,000$ which will be 1.45×10^8 in one year**

Since we have 10^{10} pages.

$$\begin{aligned}\text{Fraction needed to make a profit} &= (1.45 * 10^8)/10^{10} \\ &= 1.45/100 \\ &= 1.45\%\end{aligned}$$

Ex 28.1

1. Spatial data differs from non-spatial data in that it provides geographic context of objects by coordinates while non spatial data refers to attributes only.
2. Point data differs from region data in that it identifies discrete locations while region data represents general locations over a wider area like cities.
3. Three common kinds of spatial queries are:
 - Nearest Neighbor Queries works best when identifying close spatial object to a given point.
 - Spatial Join Queries works well when combining two spatial datasets.
 - Range Queries works well when retrieving objects in a specific range.
4. Nearest Neighbor queries are important in multimedia applications because it is used for content-based retrieval by enabling systems to find similar multimedia objects through comparison of features in both contents.
5. B+ tree index is different from spatial index in that it is mostly used for one dimensional data and is more efficient on numbers or texts queries while spatial index is used for multi-dimensional data, and it handles spatial relationships. B+ trees can be used when spatial relationships like distance or proximity are not the focus of the query while spatial index is used when trying to find objects in a specific geographic region.
6. The relationship between Z-ordering and Region Quad trees is that they are both spatial indexing techniques. Z-ordering preserves the spatial locality while the region quad trees store the spatial objects hierarchically.
7. Z-ordering and Hilbert curves as techniques to cluster spatial data has one main comparison and that is they are both space-filling curves. Z-ordering works best in linearizing the data while Hilbert curves offer better spatial locality and is mostly used when you want to retain the local topological structure of the original data as much as possible.